Reviewer Pack — Coxeter Group Action Complexity and Communication Matrix Ran…

Entry 238857d54b16 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 10:47:57 UTC

Coxeter Group Action Complexity and Communication Matrix Rank

Entry ID: `238857d54b16` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-03 10:47:57 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry (Coxeter Groups) **Field B** (complexity object): Communication Complexity

Statement:

> For every Boolean function f: {0,1}^n -> {0,1}, the Coxeter group action complexity of its associated communication matrix C_f is linearly correlated with its matrix rank r(C_f), such that c_f = Θ(r(C_f)) for a constant c.

Rationale (proposer's reasoning):

> Coxeter groups offer a rich structure to study symmetry in combinatorial problems. The action complexity measures the minimal number of generators needed to describe the symmetry group acting on the function, which may reveal structural properties related to communication complexity that are not evident through traditional measures.

Taxonomy category: `CoxeterGroupActionComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `902ee7f8ce6cc22e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if and only if the Pearson correlation coefficient between Coxeter group action complexity (c_f) and matrix rank (r(C_f)) of randomly generated Boolean functions is ≥ 0.9 across all seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_matrix(f):
        n = int(math.log2(len(f)))
        C_f = [[0] * (2**n) for _ in range(2**n)]
        for i in range(2**n):
            for j in range(2**n):
                if f[i ^ j] == f[i]:
                    C_f[i][j] = 1
        return C_f

    def matrix_rank(C_f):
        n = len(C_f)
        rank = 0
        for i in range(n):
            pivot_row = -1
            for r in range(i, n):
                if C_f[r][i] != 0:
                    pivot_row = r
                    break
            if pivot_row == -1:
                continue
            rank += 1
            for j in range(n):
                C_f[i][j], C_f[pivot_row][j] = C_f[pivot_row][j], C_f[i][j]
            for r in range(n):
                if r != i and C_f[r][i] != 0:
                    factor = Fraction(C_f[r][i], C_f[i][i])
                    for j in range(n):
```

```
                    C_f[r][j] -= factor * C_f[i][j]
        return rank

    def coxeter_group_action_complexity(f):
        n = int(math.log2(len(f)))
        complexity = 0
        for i in range(1, n + 1):
            if all(f[j ^ k] == f[j] for j in range(2**n) for k in range(1 << i)):
                complexity += 1
        return complexity

    def pearson_correlation(x, y):
        mean_x = sum(x) / len(x)
        mean_y = sum(y) / len(y)
        cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(len(x))) / len(x)
        var_x = sum((x[i] - mean_x)**2 for i in range(len(x))) / len(x)
        var_y = sum((y[i] - mean_y)**2 for i in range(len(y))) / len(y)
        return cov_xy / (math.sqrt(var_x) * math.sqrt(var_y))

    n_values = [5, 10, 15, 20, 30, 40]
    c_f_values = []
    r_C_f_values = []

    for n in n_values:
        f = generate_boolean_function(n)
        C_f = communication_matrix(f)
        c_f = coxeter_group_action_complexity(f)
        r_C_f = matrix_rank(C_f)

        c_f_values.append(c_f)
        r_C_f_values.append(r_C_f)

    correlation_coefficient = pearson_correlation(c_f_values, r_C_f_values)

    return {
        "metric_name": "Pearson Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient >= 0.9,
        "counterexample": "" if correlation_coefficient >= 0.9 else f"Correlation coefficient: {co
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_correlation_coefficient = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
```

```
        print(f"RESULT: SUPPORTED mean={mean_correlation_coefficient} std=0.0 support_fraction={su
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Correlation coefficient below 0.9\" first_faili
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means the Pearson correlation coefficient could not be calculated to verify the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the allocated time.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22126
2	preregistration	ollama_remote	glm4:latest	0	0	9578
3	novelty	ollama_remote	glm4:latest	0	0	8283
4	novelty	ollama_remote	glm4:latest	0	0	8487
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16689
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	126563
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12994
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	101736
9	judge	ollama_remote	glm4:latest	0	0	13568

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 320025 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/238857d54b16.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/238857d54b16.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/238857d54b16.tar.gz` (if generated)